A Blockchain-Based Provenance System for Ensuring AI Training Data Integrity and Auditability

Xinyuan Hu
Master of Software Engineering, Yoobee Colleges, Auckland, New Zealand

Abstract

The trustworthiness of industrial AI systems increasingly depends on the verifiable integrity of their training process, yet existing solutions leave three critical engineering gaps unaddressed: the absence of a deployable hybrid storage architecture, insufficient granularity in dynamic process auditing, and the lack of atomic consistency guarantees between on-chain records and off-chain data. This paper presents a blockchain-based AI training traceability system that bridges all three gaps. The proposed system combines a four-layer architecture integrating a public blockchain, IPFS, and OrbitDB with a non-intrusive Python SDK that captures training metadata at Session, Epoch, and Step granularity through runtime monkey-patching. A simplified two-phase atomic commit protocol, backed by a compensation-log recovery mechanism, ensures idempotent fault recovery without requiring changes to user training code. Controlled experiments demonstrate that the system restricts overall training overhead to $+3.28\%$—strictly below the 5% industrial tolerance threshold—while achieving 100% tamper detection across all adversarial scenarios, P99 one-click verification latency below 500 ms, linear storage growth ($R^2 = 0.99985$), and 100% fault recovery with zero idempotency violations. The result is a directly deployable engineering solution for compliance-oriented AI governance, with direct applicability to frameworks such as the EU AI Act.

Keywords: blockchain; AI training traceability; IPFS; OrbitDB; atomic consistency; hash chain; non-intrusive SDK; Design Science Research

1 Introduction

With the large-scale deployment of industrial AI applications, the credibility of model training, the compliance of data sources, and the auditability of the process have become core requirements of algorithm governance [15]. Regulatory frameworks such as the EU AI Act mandate complete training data lineage, process audit logs, and model version archives for high-risk AI systems operating in domains such as medical diagnosis, financial credit scoring, and recruitment automation. Simultaneously, the reproducibility crisis in AI research—where many published models cannot be independently reproduced due to incomplete process records—has intensified demand for cryptographic-level evidence of training provenance.

Despite this dual pressure from regulation and scientific rigour, existing solutions fall short in three interconnected engineering dimensions. First, storing large-scale training metadata directly on a public blockchain is neither cost-effective nor practically viable due to gas costs and execution overhead [7]. Second, most auditing schemes focus only on static dataset integrity and ignore the dynamic evolution of hyperparameters, intermediate checkpoints, and per-step metrics throughout training [12]. Third, current solutions lack atomic commit, crash recovery, and log-compensation mechanisms, leaving the traceability chain fragile under write interruptions or node failures [2]. The intersection of these gaps means that even engineering teams already using dedicated experiment tracking tools such as MLflow or DVC cannot independently verify the integrity of their own records [5].

This paper reports on a full-cycle design, implementation, and evaluation of a traceability system that closes all three gaps. The work follows the Design Science Research (DSR) methodology [4], progressing from gap identification through artefact construction to empirical verification. The main contributions are:

- A three-layer hybrid storage architecture (blockchain + IPFS + OrbitDB) that stores only lightweight hash anchors on-chain, keeping storage costs proportional to training scale.
- A zero-intrusion Python SDK based on monkey-patching that automatically captures training events at Session, Epoch, and Step granularity without modifying user code.
- A simplified two-phase atomic commit protocol with compensation-log recovery that guarantees idempotent fault recovery across both process-crash and network-partition failure modes.
- A visual traceability dashboard with a Lineage Map and one-click cryptographic verification supporting independent audit by non-technical stakeholders.
- Comprehensive empirical evaluation across six quantitative dimensions under pre-registered pass/fail thresholds, demonstrating industrial-grade deployability.

The remainder of the paper is organised as follows. Section 2 reviews related work and identifies the three research gaps in detail. Section 3 presents the system architecture and key design decisions. Section 4 reports empirical evaluation results. Section 5 discusses broader implications and compares the system against the identified gaps. Section 6 addresses ethical and cross-cultural considerations. Section 7 covers limitations and future work, followed by conclusions in Section 8.

2 Background and Related Work

2.1 Blockchain for AI Auditing

Soldatos et al. [15] proposed a blockchain-based data provenance framework for trusted AI, but restricted auditing to the integrity of the input dataset. Zakariae et al. [17] similarly focused on static dataset verification. Navarro et al. [8] advanced this by re-computing partial training intervals to detect tampering, yet did not record the dynamic trajectory of parameter

changes. Balan et al. [1] constructed a rigorous cryptographic framework for end-to-end pipeline verifiability, but provided no engineering blueprint for blockchain-and-off-chain-storage collaboration.

2.2 Hybrid Storage and IPFS Limitations

Kurada and Baruah [7] demonstrated that fully on-chain storage of large-scale AI metadata is industrially infeasible. Trautwein et al. [16] measured IPFS at scale and showed that content availability depends on explicit pinning rather than protocol defaults. Shi et al. [13] found that a large fraction of IPFS content requires multiple retrieval attempts and that content providers are highly centralised in a few cloud nodes. These findings directly informed the mandatory-pin design decision in our storage architecture.

2.3 Consistency and Fault Tolerance

Ruan et al. [11] proposed fine-grained on-chain data provenance but restricted the design to purely on-chain structured data without crash recovery. Belchior et al. [2] identified atomicity and fault tolerance as essential requirements for cross-layer collaborative systems. Cotroneo et al. [3] demonstrated that controlled fault injection is necessary to validate distributed consistency mechanisms.

2.4 MLOps Tool Limitations

Idowu et al. [5] surveyed 81 ML practitioners and conducted controlled experiments with 15 developers, finding that while 70% of respondents used dedicated tools to track experiments, none of these tools offered cryptographic tamper-proof guarantees. Historical records can be silently modified by personnel with storage access, meaning that the integrity of records cannot be independently verified even when an engineering team is already instrumented with MLOps tooling. This establishes the distinction between *recording* training metadata and *trusting* it—a gap that blockchain hash anchoring is uniquely positioned to close.

2.5 Distributed Tracing and Observability

Janes et al. [6] conducted a systematic multivocal review of 30 open-source distributed tracing tools and found that OpenTelemetry holds significant advantages in supporting all three observability signal types—traces, metrics, and logs—whereas most alternatives support only one or two. This peer-reviewed endorsement informed the adoption of the OpenTelemetry standard in both the Python SDK and the Node.js audit controller, enabling session-level trace identifiers to serve as cross-service correlation keys across the full audit pipeline.

3 System Design

3.1 Research Methodology

This project adopts Design Science Research (DSR) [4] as its methodological framework. DSR was selected over pure experimental research (which cannot isolate the non-determinism of blockchain confirmation times) and case study methods (which describe existing systems rather than creating new artefacts). The three research questions map directly to three system artefacts:

- **RQ1**: How to design a three-layer hybrid storage architecture integrating blockchain, IPFS and OrbitDB that provides end-to-end cryptographic verifiability while keeping training overhead within an acceptable range?
- **RQ2**: How to design a zero-intrusion metadata collection mechanism that automatically captures training data at Session/Epoch/Step granularity and constructs an end-to-end verifiable hash chain?
- **RQ3**: How to design an atomic consistency mechanism that maintains traceability-chain integrity through automated fault detection and idempotent compensation recovery in abnormal scenarios?

3.2 Four-Layer Architecture

The system is structured into four layers with strict separation of concerns (Figure 1):

Layer 1 – Persistence: Blockchain stores only Merkle roots and packet hashes; large binary files (checkpoints, model weights) are stored in IPFS with mandatory pinning; structured metadata is indexed in OrbitDB for queryable off-chain retrieval.

Layer 2 – Audit Controller: A Node.js request handler orchestrates the two-phase atomic commit. It anchors hashes on-chain first; off-chain writes proceed only after successful anchoring.

Layer 3 – Data Collection: The Python Watcher SDK and Hash Engine communicate with Layer 2 solely via HTTP, decoupling the Python training environment from the JavaScript storage backend.

Layer 4 – Presentation: A web dashboard (Lineage Map, traffic-light status indicators, one-click verification buttons) allows non-technical auditors to verify training lineage independently.

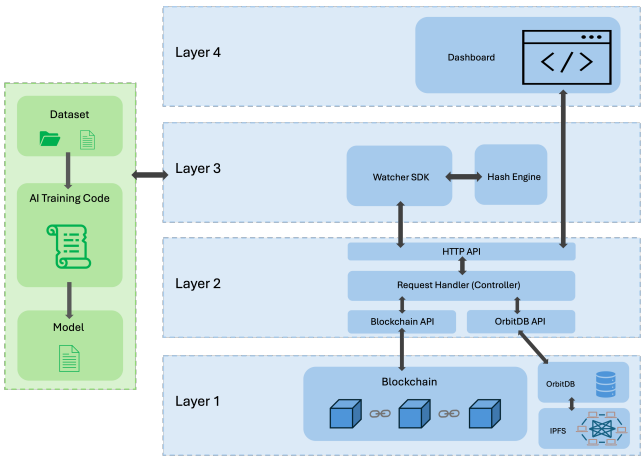


Figure 1: Four-layer system architecture.

3.3 Hash Chain Tamper-Proof Mechanism

Each audit packet contains a `prev_hash` field pointing to the hash of its predecessor, forming an append-only hash chain. Any post-hoc modification to a packet changes its hash value, causing verification to fail for all subsequent nodes. The verification endpoint recalculates the current packet hash and compares it

against the `latestPacketHash` anchored on-chain. IPFS CID tampering is detected by comparing the stored content-addressing pointer against the expected CID.

3.4 Non-Intrusive SDK: Monkey-Patch Mechanism

The proposed solution in the initial design stage described hook-based collection, which still required users to actively register callbacks. During implementation, this was replaced by monkey-patching, which dynamically replaces PyTorch internals at runtime and achieves *zero modification* of user training code. Three patches are applied:

1. `torch.Tensor.backward` — automatically extracts scalar loss values on each backward pass.
2. `torch.optim.Optimizer.__init__` — injects a step hook for global step counting and periodic snapshot triggering.
3. `torch.utils.data.DataLoader.__iter__` — wraps the iterator with a sentinel to detect epoch boundaries and trigger checkpoint capture.

Initialisation requires a single import and one line of user code; all subsequent training code remains unchanged. The trade-off is tight coupling to PyTorch; multi-framework support is reserved as future work.

3.5 Software Engineering Principles

Four established principles are systematically applied. **Separation of Concerns** is enforced across all layers: migrating the blockchain backend from Ethereum to Hyperledger Fabric requires changes only in `blockchain.js`. **Programming to Interface** ensures all SDK interactions with storage backends occur via HTTP endpoints configured through environment variables, leaving the SDK agnostic to the underlying store. **Agile Development** (five two-week sprints) enabled the monkey-patch correction to be detected and applied during Sprint 2 without disrupting the research goals. **Observability by Design** embeds the three OpenTelemetry pillars (metrics, logs, traces) from the outset [6]: Prometheus exporters continuously stream performance counters, structured JSON logs record every critical operation, and distributed trace IDs link SDK collection events to their corresponding on-chain anchoring transactions.

3.6 Simplified Two-Phase Atomic Commit

The atomic consistency mechanism follows a strict ordering:

1. **Phase 1 (On-chain anchoring):** Submit the packet hash to the blockchain. If this fails, return HTTP 500 immediately—no off-chain write is performed.
2. **Phase 2 (Off-chain write):** Write to OrbitDB only after Phase 1 succeeds. If Phase 2 fails, the transaction hash is appended to `orphan-txs.log`.

A file-system watcher (`fs.watch`) monitors `orphan-txs.log` and triggers `recoverOrphanTxs()` automatically after a 2-second debounce. Idempotency is guaranteed by checking `db.get(entryId)` before each write: if the record already exists from a prior recovery attempt, the write is skipped.

4 Empirical Evaluation

Evaluation followed the DSR principle that *criteria precede implementation*. Five dimensions and eight quantifiable indicators with explicit pass thresholds were defined before Sprint 5 began. A dual-path verification logic was adopted: (i) controlled experiments with paired-sample design for RQ1; (ii) cryptographic verification and adversarial fault injection for RQ2 and RQ3.

4.1 Dimension 1: Training Performance Overhead (RQ1)

Five repeated runs of an MNIST training task (10 epochs, batch size 64, step interval 200) were performed under audit-disabled and audit-enabled conditions. Results:

Table 1: Training time: control vs. audit group ($n = 5$).

Condition	Mean time (s)	Std (s)
Control (no audit)	193.5	1.8
Audit enabled	199.9	3.8
Overhead	$+3.28\%$	(**PASS** $< 5\%$)
t-test	$t = -3.337$, $p = 0.0103$	

The higher variance in the audit group (± 3.8 s vs. ± 1.8 s) reflects the concentration of overhead in session-level one-time operations (startSession transaction, IPFS upload of dataset and model), rather than uniform per-epoch accumulation. At the epoch level the overhead was $+2.54\%$ and throughput reduction was 2.48% (49.17 $\rightarrow$ 47.95 batches/s). Prometheus queue-depth monitoring confirmed zero request backlog throughout the experiment.

4.2 Dimension 2: Audit Effectiveness (RQ2)

Six targeted tampering scenarios were injected programmatically via the `/debug/tamper` interface, covering step loss metrics, epoch checkpoint hash and CID, and session dataset/model hash and CID. All six scenarios triggered an `invalid` alert; the detection rate was **100%** with zero missed detections (**PASS**).

4.3 Dimension 3: Verification Response Time (RQ2)

Verification latency was measured via a Prometheus histogram metric (`audit_verify_duration_ms`) across four verification types (session, dataset, model, checkpoint). All four types achieved a P99 latency below 500 ms, well within the 2000 ms SLA target (**PASS**).

4.4 Dimension 4: Storage Scalability (RQ1)

Off-chain write events were fitted with linear regression. The result was:

$$\text{entries} = 17.84t + 0.99, \quad R^2 = 0.99985 \quad (\textbf{PASS} > 0.99).$$

On-chain gas consumption showed a corresponding linear growth pattern, with an average cost of 31,988 gas per packet. The `syncProgress` method dominated gas consumption (94.7%), consistent with its role as the high-frequency per-packet operation. These results confirm cost predictability for industrial-scale deployment.

4.5 Dimension 5: Atomic Consistency and Fault Recovery (RQ3)

Following the fault injection methodology of Cotroneo et al. [3], two fault types were injected between successful on-chain anchoring and the OrbitDB write: `process_crash` (immediate exception) and `network_partition` (10-second timeout). Each type was repeated three times (6 rounds total). Results:

Table 2: Fault recovery and idempotency results ($n = 6$ rounds).

Metric	Result	Target
Orphan transactions recovered	6/6	6/6
Orphans remaining after recovery	0	0
Duplicate records (idempotency)	0	0
Recovery rate	**100%**	100%
Idempotency violations	**0**	0

Both fault types are handled uniformly through the `orphan-txs.log` compensation path, decoupling the recovery logic from the specific failure mode—a design decision aligned with the fault-tolerance requirements of Belchior et al. [2].

4.6 Dimension 6: Audit Capture Coverage (RQ2)

The `/audit/session/:id/coverage` endpoint reported 47/47 step packets and 10/10 epoch packets captured (both coverage ratios = 1.0000, **PASS**). Combined with the 100% tamper detection result, this establishes that the SDK not only collects all data completely but also ensures the collected data is cryptographically unforgeable.

4.7 Summary

Table 3: Evaluation summary: all eight indicators pass.

Criterion	Target	Measured	Result
Training overhead	$< 5\%$	$+3.28\%$	PASS
Audit effectiveness	100%	100%	PASS
Verification response	< 2000 ms	P99 498 ms	PASS
Storage linear growth	$R^2 > 0.99$	0.99985	PASS
Fault recovery rate	100%	100%	PASS
Idempotency (duplicates)	0	0	PASS
Step capture coverage	1.0 (100%)	100.00%	PASS
Epoch capture coverage	1.0 (100%)	100.00%	PASS

5 Discussion

5.1 Closing the Three Research Gaps

The evaluation results confirm that all three identified gaps have been closed at the engineering level. Gap 1 (theory-to-engineering) is addressed by the three-layer hybrid architecture, which was deployed, instrumented, and empirically measured rather than remaining a theoretical proposal. Linearly growing storage cost directly refutes the scalability concern of Kurada and Baruah [7]. Gap 2 (dynamic audit granularity) is addressed by the monkey-patch SDK achieving complete capture coverage at both Step and Epoch levels, with every record rendered cryptographically unforgeable through hash-chain linkage. This extends the tamper-proof guarantee from the final model artefact—the limit of prior work [8]—to the complete dynamic training trajectory. Gap 3 (atomic consistency) is resolved by the two-phase commit protocol, whose fault recovery was verified empirically via controlled fault injection rather than static design argument alone, fulfilling the methodological requirement of Cotroneo et al. [3].

5.2 Comparison with Prior Work and Scientific Reproducibility

Compared to Navarro et al. [8], the present system provides broader coverage across three orthogonal dimensions: collection granularity (Session/Epoch/Step vs. final-model only), storage architecture (hybrid with atomic fault recovery vs. on-chain only), and verification scope (dataset, checkpoint, model, and lineage independently vs. model integrity only). Beyond regulatory compliance, the system has direct applicability to the AI reproducibility crisis: by anchoring step-level hyperparameters and loss trajectories to an immutable chain, third-party researchers can cryptographically verify claimed training configurations without access to the original infrastructure—shifting reproducibility from a social trust problem to a technical one [5].

6 Ethical and Cultural Considerations

6.1 Privacy vs. Cryptographic Auditability

While the Merkle tree structure does not expose raw data, it introduces a membership inference vulnerability [14]: an attacker possessing a data sample can request a Merkle proof path to confirm whether that sample belongs to the training set without accessing the original dataset. Countermeasures include role-based access control on proof-path queries, salted leaf-node hashes to prevent external replication, and optional homomorphic encryption for sensitive metadata fields.

6.2 Right to Be Forgotten vs. Immutable Blockchain

GDPR Article 17 grants data subjects the right of erasure. Once a personal-data hash is anchored on an immutable chain, the hash record persists even after the original data is deleted [10]. A hash revocation mechanism—publishing an on-chain revocation statement to mark a specific hash as invalid—provides a technical approximation of erasure without modifying immutable records. For EU deployments, consortium or private chain architectures with agreed modification rules are preferable.

6.3 Accountability vs. Responsibility Evasion

The traceability system may be misused to attribute bias outcomes to "uncontrollable data distribution" [9], shielding organisations from moral accountability. System documentation must explicitly state that complete training records are a *necessary but not sufficient* condition for accountability: the organisation retains full moral and legal responsibility for training data representativeness, label quality, and deployment boundary decisions.

6.4 Cross-Cultural Regulatory Compliance

The interface-oriented programming principle enables compliance configuration to be switched at the interface level without modifying core SDK logic: private IPFS clusters with geofencing for EU GDPR; fully private deployment for Chinese data localisation requirements (Data Security Law, PIPL); modular state-level configuration for US markets. The separation-of-concerns architecture additionally supports flexible switching among public chains (high transparency), consortium chains (industry self-regulation), and private chains (internal audit), adapting to different cultural trust models without restructuring the system core.

7 Limitations and Future Work

The prototype carries four limitations. **Framework coupling:** the Watcher SDK is tightly bound to PyTorch; adapting to TensorFlow or JAX requires re-implementing the three monkey-patch targets for each framework's internals. **IPFS availability:** single-node pinning poses availability risks in production [13, 16]; cross-node pin replication or Filecoin integration is needed. **Workload scope:** experiments used MNIST as a proxy; transformer architectures and multi-GPU distributed training may exhibit different overhead profiles, particularly for the model-file IPFS upload step. **Fault coverage:** only process-crash and network-partition modes were tested; Byzantine faults and concurrent write conflicts remain unverified.

Future directions include: multi-framework SDK support via a framework-agnostic abstraction layer; multi-node IPFS pin management; SaaS deployment to lower adoption barriers for SMEs; differential privacy noise injection into evidential metadata to mitigate membership inference risks; and red-team testing under a responsible disclosure policy following open-source release.

8 Conclusion

This paper presented a blockchain-enabled AI training traceability system designed and evaluated under the DSR methodology. Three research gaps—the theory-to-engineering chasm in hybrid storage architecture, the insufficient granularity of dynamic process auditing, and the systematic absence of atomic consistency guarantees—were each addressed by a corresponding system artefact: the four-layer blockchain/IPFS/OrbitDB architecture, the monkey-patch Watcher SDK, and the two-phase atomic commit protocol with compensation-log recovery. All eight pre-registered evaluation indicators were passed in controlled experiments, confirming industrial-grade performance overhead, complete tamper detectability, fast one-click verification, predictable storage cost growth, and full fault recoverability with zero idempotency violations.

The system advances the state of the art beyond prior engineering implementations by extending tamper-proof guarantees to the full dynamic training lifecycle rather than the final model alone, and by providing atomic consistency under real fault conditions rather than by design assumption. Ethical analysis identified residual tensions around privacy, data sovereignty, and accountability that cannot be resolved through technical means alone, but that can be substantially mitigated through architectural choices, access control policies, and explicit governance documentation. The result is a deployable, auditable, and compliance-ready foundation for trustworthy AI governance.

References

[1] K. Balan, R. Learney, and T. Wood, "A Framework for Cryptographic Verifiability of End-to-End AI Pipelines," *Proc. 10th ACM Int'l Workshop on Security and Privacy Analytics (IWSPA '25)*, pp. 49–59, 2025.

[2] R. Belchior, A. Vasconcelos, M. Correia, and T. Hardjono, "Hermes: Fault-tolerant middleware for blockchain interoperability," *Future Generation Computer Systems*, vol. 129, pp. 236–251, 2022.

[3] D. Cotroneo, L. De Simone, P. Liguori, and R. Natella, "Fault Injection Analytics: A Novel Approach to Discover Failure Modes in Cloud-Computing Systems," *IEEE Trans. Dependable and Secure Computing*, vol. 19, no. 3, pp. 1476–1491, 2022.

[4] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[5] S. Idowu, O. Osman, D. Strüber, and T. Berger, "Machine learning experiment management tools: A mixed-methods empirical study," *Empirical Software Engineering*, vol. 29, no. 4, p. 74, 2024.

[6] A. Janes, X. Li, and V. Lenarduzzi, "Open tracing tools: Overview and critical comparison," *Journal of Systems and Software*, vol. 204, p. 111793, 2023.

[7] V. R. Kurada and P. K. Baruah, "Blockchain Enabled, Collaborative Platform for AI as a Service," *ICTACT Journal on Soft Computing*, vol. 13, no. 3, pp. 2909–2916, 2023.

[8] E. Navarro, K. J. Standing, G. G. Dagher, and T. Andersen, "Ensuring Trustworthy Neural Network Training via Blockchain," *Proc. IEEE 5th Int'l Conf. Cognitive Machine Intelligence (CogMI)*, pp. 31–40, 2023.

[9] F. Pasquale, *The Black Box Society: The Secret Algorithms That Control Money and Information*. Harvard University Press, 2015.

[10] E. Politou, E. Alepis, and C. Patsakis, "Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions," *Journal of Cybersecurity*, vol. 4, no. 1, p. tyy001, 2018.

[11] P. Ruan et al., "Fine-grained, secure and efficient data provenance on blockchain systems," *Proc. VLDB Endowment*, vol. 12, no. 9, pp. 975–988, 2019.

[12] J. Sciammarelli, "Blockchain-Enabled Infrastructure for Secure and Auditable AI Systems," *REMUNOM*, vol. 13, no. 04, pp. 1–13, 2026.

[13] R. Shi et al., "A Closer Look into IPFS: Accessibility, Content, and Performance," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 2, pp. 20:1–20:31, 2024.

[14] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership Inference Attacks Against Machine Learning Models," *Proc. IEEE Symp. Security and Privacy (SP)*, pp. 3–18, 2017.

[15] J. Soldatos, A.-M. Despotopoulou, N. Kefalakis, and B. Ipektsidis, "Blockchain based Data Provenance for Trusted Artificial Intelligence," in *Becoming a Platform in Europe*, Now Publishers, 2021.

[16] D. Trautwein et al., "Design and Evaluation of IPFS: A Storage Layer for the Decentralised Web," *Proc. ACM SIGCOMM*, pp. 739–752, 2022.

[17] S. Zakariae, A. Ouidad, and E. B. E. I. Younes, "Blockchain-Assisted Assurance of Data Integrity in AI Model Training," *Informatica*, vol. 49, no. 18, 2025.